Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-211БВ-25

Студент: Щепинов Д.С.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 01.11.26

Москва, 2026

Постановка задачи

Вариант 6.

Произвести перемножение 2-ух матриц, содержащих комплексные числа

Общий метод и алгоритм решения

- pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine) (void *), void *arg) — создает новый поток исполнения.
- pthread_join(pthread_t thread, void **retval) — блокирует вызывающий поток до завершения указанного потока.
- pthread_self() — возвращает идентификатор текущего потока (используется неявно внутри логики).
- calloc(size_t nmemb, size_t size) — выделяет и обнуляет память под массивы.
- timespec_get(struct timespec *ts, int base) —функция получения текущего времени с наносекундной точностью.

Описание алгоритма и структур данных
Последовательный алгоритм представляет собой классический тройной цикл (сложность $O(N^3)$): перебираются строки i первой матрицы, столбцы j второй, и внутри суммируются произведения элементов k.

Параллельный алгоритм использует разбиение по строкам. Главный поток создает T рабочих потоков. Каждому потоку передается структура ThreadArg, содержащая указатели на матрицы и диапазон строк, которые он должен обработать.
Поскольку каждый поток записывает результат строго в свои строки итоговой матрицы C, состояние гонки отсутствует, и использование мьютексов не требуется. Потоки только читают общие матрицы A и B, что безопасно.

Код программы

main.c

```
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <time.h>

#include <math.h>

#include <string.h>

#include <errno.h>

#include <limits.h>



typedef struct {

    double re;
```

```
    double im;
} Complex;


typedef struct {
    Complex *data;
    int rows;
    int cols;
} Matrix;


int parse_int(const char *str, const char *name, int min_val, int max_val, int *out) {
    char * endptr;
    errno = 0;
    long val = strtol(str, &endptr, 10);


    if (errno == ERANGE || val < INT_MIN || val > INT_MAX) {
        fprintf(stderr, "Ошибка: %s вне диапазона int\n", name);
        return 0;
    }
    if (*endptr != '\0') {
        fprintf(stderr, "Ошибка: %s — не число ('%s')\n", name, str);
        return 0;
    }
    if (val < min_val || val > max_val) {
        fprintf(stderr, "Ошибка: %s должно быть в диапазоне [%d, %d], получено %ld\n", name, min_val,
max_val, val);
        return 0;
    }
    *out = (int)val;
    return 1;
}


double get_time_sec() {
    struct timespec ts;
    timespec_get(&ts, TIME_UTC);
    return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;
}


typedef struct {
```

```
    const Matrix *A;
    const Matrix *B;
    Matrix *C;
    int start_row;
    int end_row;
    int thread_id;
} ThreadArg;

Matrix* matrix_create(int rows, int cols) {
    Matrix *m = (Matrix*)malloc(sizeof(Matrix));
    if (!m) {
        fprintf(stderr, "Ошибка выделения памяти под матрицу\n");
        exit(1);
    }

    m->rows = rows;
    m->cols = cols;

    m->data = (Complex*)calloc(rows * cols, sizeof(Complex));
    if (!m->data) {
        fprintf(stderr, "Ошибка выделения памяти под данные матрицы\n");
        free(m);
        exit(1);
    }

    return m;
}

void matrix_free(Matrix *m) {
    if (m) {
        free(m->data);
        free(m);
    }
}

void matrix_random_data(Matrix *m) {
    for (int i = 0; i < m->rows * m->cols; i++) {
```

```
        m->data[i].re = (double)(rand() % 100) / 10.0;
        m->data[i].im = (double)(rand() % 100) / 10.0;
    }
}

int verify_results(const Matrix *C1, const Matrix *C2) {
    double epsilon = 1e-6;
    for (int i = 0; i < C1->rows * C1->cols; i++) {
        if (fabs(C1->data[i].re - C2->data[i].re) > epsilon ||
            fabs(C1->data[i].im - C2->data[i].im) > epsilon) {
            return 0;
        }
    }
    return 1;
}

Complex complex_mul(Complex a, Complex b) {
    Complex res;
    res.re = a.re * b.re - a.im * b.im;
    res.im = a.re * b.im + a.im * b.re;
    return res;
}

double multiply_without_threads(const Matrix *A, const Matrix *B, Matrix *C) {
    double start_time = get_time_sec();

    for (int i = 0; i < A->rows; i++) {
        for (int j = 0; j < B->cols; j++) {
            Complex sum = {0.0, 0.0};
            for (int k = 0; k < A->cols; k++) {
                Complex a = A->data[i * A->cols + k];
                Complex b = B->data[k * B->cols + j];
                Complex prod = complex_mul(a, b);
                sum.re += prod.re;
                sum.im += prod.im;
            }
            C->data[i * C->cols + j] = sum;
```

```
        }
    }

    return get_time_sec() - start_time;
}

void* thread_func(void *arg) {
    ThreadArg *data = (ThreadArg*)arg;

    const Matrix *A = data->A;
    const Matrix *B = data->B;
    Matrix *C = data->C;

    for (int i = data->start_row; i < data->end_row; i++) {
        for (int j = 0; j < B->cols; j++) {
            Complex sum = {0.0, 0.0};

            for (int k = 0; k < A->cols; k++) {
                Complex a = A->data[i * A->cols + k];
                Complex b = B->data[k * B->cols + j];

                Complex prod = complex_mul(a, b);
                sum.re += prod.re;
                sum.im += prod.im;
            }

            C->data[i * C->cols + j] = sum;
        }
    }

    return NULL;
}

double multiply_with_threads(const Matrix *A, const Matrix *B, Matrix *C, int num_threads) {
    if (num_threads > C->rows) num_threads = C->rows;

    pthread_t *threads = (pthread_t*)malloc(num_threads * sizeof(pthread_t));
```

```
    ThreadArg *args = (ThreadArg*)malloc(num_threads * sizeof(ThreadArg));

    int rows_per_thread = C->rows / num_threads;
    int remainder = C->rows % num_threads;
    int current_row = 0;

    double start_time = get_time_sec();

    for (int i = 0; i < num_threads; i++) {
        args[i].A = A;
        args[i].B = B;
        args[i].C = C;
        args[i].thread_id = i;

        args[i].start_row = current_row;
        int rows_to_process = rows_per_thread + (i < remainder ? 1 : 0);
        args[i].end_row = current_row + rows_to_process;
        current_row = args[i].end_row;

        if (pthread_create(&threads[i], NULL, thread_func, &args[i]) != 0) {
            perror("Ошибка создания потока");
            exit(1);
        }
    }

    for (int i = 0; i < num_threads; i++) {
        pthread_join(threads[i], NULL);
    }

    double end_time = get_time_sec();

    free(threads);
    free(args);

    return end_time - start_time;
}
```

```
void create_table_results(Matrix *A, Matrix *B, Matrix *C_1, Matrix *C_2) {
    // Список количества потоков для тестирования
    int thread_counts[] = {1, 2, 4, 8, 16, 32, 128, 1024};
    int num_counts = sizeof(thread_counts) / sizeof(thread_counts[0]);
    int runs = 3; // Количество прогонов для стабильности

    // 1. Замеряем последовательную версию (3 раза, берем лучший результат)
    double t_seq_min = 1e9;
    for (int r = 0; r < runs; r++) {
        double t = multiply_without_threads(A, B, C_1);
        if (t < t_seq_min) t_seq_min = t;
    }

    printf("Потоков\t\tВремя (мс)\tУскорение\tЭффективность\n");

    printf("Послед\t\t%.2f\t\t1.00\t\t100.0%%\n", t_seq_min * 1000.0);

    for (int i = 0; i < num_counts; i++) {
        int threads = thread_counts[i];
        double t_par_min = 1e9;

        for (int r = 0; r < runs; r++) {
            double t = multiply_with_threads(A, B, C_2, threads);
            if (t < t_par_min) t_par_min = t;
        }

        if (!verify_results(C_1, C_2)) {
            printf("(!) ОШИБКА: Результаты не совпадают!\n");
        }

        double speedup = t_seq_min / t_par_min;
        double efficiency = speedup / threads;

        printf("%d\t\t%.2f\t\t%.2f\t\t%.1f%%\n", threads, t_par_min * 1000.0, speedup, efficiency * 100.0);
    }
}
```

```
int main(int argc, char *argv[]) {
    if (argc == 5) {
        int M, K, N, threads;

        if(!parse_int(argv[1], "M", 1, 20000, &M)) return 1;
        if(!parse_int(argv[2], "K", 1, 20000, &K)) return 1;
        if(!parse_int(argv[3], "N", 1, 20000, &N)) return 1;
        if(!parse_int(argv[4], "threads", 1, 100000, &threads)) return 1;

        Matrix *A = matrix_create(M, K);
        Matrix *B = matrix_create(K, N);
        Matrix *C_1 = matrix_create(M, N);
        Matrix *C_2 = matrix_create(M, N);

        srand(12345);
        matrix_random_data(A);
        matrix_random_data(B);

        double t_seq = multiply_without_threads(A, B, C_1);
        double t_par = multiply_with_threads(A, B, C_2, threads);

        if (verify_results(C_1, C_2)) {
            printf("Результаты совпадают!\n");
        } else {
            printf("ОШИБКА: Результаты не совпадают!\n");
        }

        printf("Время (seq): %.4f сек\n", t_seq);
        printf("Время (par): %.4f сек\n", t_par);

        double speedup = t_seq / t_par;
        double efficiency = speedup / threads;

        printf("Ускорение (S) = %.2f\n", speedup);
        printf("Эффективность (X) = %.1f%%\n", efficiency * 100.0);

        matrix_free(A); matrix_free(B); matrix_free(C_1); matrix_free(C_2);
```

```
        return 0;
    }

    if (argc == 4) {
        int M, K, N, threads;

        if(!parse_int(argv[1], "M", 1, 20000, &M)) return 1;
        if(!parse_int(argv[2], "K", 1, 20000, &K)) return 1;
        if(!parse_int(argv[3], "N", 1, 20000, &N)) return 1;

        Matrix *A = matrix_create(M, K);
        Matrix *B = matrix_create(K, N);
        Matrix *C_1 = matrix_create(M, N);
        Matrix *C_2 = matrix_create(M, N);

        srand(12345);
        matrix_random_data(A);
        matrix_random_data(B);

        create_table_results(A, B, C_1, C_2);

        matrix_free(A);
        matrix_free(B);
        matrix_free(C_1);
        matrix_free(C_2);

        return 0;
    }

    fprintf(stderr, "Использование:\n");
    fprintf(stderr, "%s [M] [K] [N] [threads] - один замер с заданным числом потоков\n", argv[0]);
    fprintf(stderr, "%s [M] [K] [N] - автоматическая таблица\n", argv[0]);
    return 1;

    return 0;
}
```

Протокол работы программы

```
lenovo@DESKTOP-T92L30D:/mnt/c/Users/Lenovo/OneDrive/Документы/GitHub/OS_2026/lab2$ ./main.out 500 500 500
Потоков         Время (мс)      Ускорение       Эффективность
Послед          247.93          1.00            100.0%
1               215.06          1.15            115.3%
2               136.17          1.82            91.0%
4               85.53           2.90            72.5%
8               54.06           4.59            57.3%
16              31.39           7.90            49.4%
32              34.57           7.17            22.4%
128             35.51           6.98            5.5%
1024            70.46           3.52            0.3%
```

Strace результаты

```
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 28.83    0.299286         379       789           clone3
 26.47    0.274787          87      3157           rt_sigprocmask
  9.03    0.093731         118       789           madvise
  7.68    0.079752         100       790           rseq
  7.25    0.075303         104       724           mmap
  7.08    0.073550         102       715           mprotect
  6.24    0.064798          82       790           set_robust_list
  6.10    0.063321          88       713           munmap
  1.25    0.012929         680        19           futex
  0.04    0.000388         388         1           write
  0.01    0.000081          27         3           brk
  0.01    0.000055          18         3           fstat
  0.00    0.000050          25         2           close
  0.00    0.000046          46         1           getrandom
  0.00    0.000042          42         1           set_tid_address
  0.00    0.000041          41         1           prlimit64
  0.00    0.000039          39         1           arch_prctl
  0.00    0.000018          18         1           rt_sigaction
  0.00    0.000000           0         1           read
  0.00    0.000000           0         2           pread64
  0.00    0.000000           0         1         1 access
  0.00    0.000000           0         1           execve
  0.00    0.000000           0         2           openat
------ ----------- ----------- --------- --------- ----------------
100.00    1.038217         122      8507         1 total
```

Вывод

В ходе лабораторной работы были приобретены навыки управления потоками с использованием POSIX Threads. Была успешно реализована параллельная версия перемножения матриц комплексных чисел, использующая декомпозицию по строкам, что позволило избежать использования мьютексов и накладных расходов на синхронизацию.

Анализ замеров (на матрицах 500x500 на 16-ядерном процессоре) что, пик ускорения достигается ровно на 16 потоках, что соответствует количеству логических ядер системы. При увеличении количества потоков до 32, 128 и 1024 наблюдается падение ускорения и резкое снижение эффективности. Это является проявлением псевдопараллелизма: операционная система

тратит слишком много процессорного времени на создание потоков и переключение контекста между ними, вместо того чтобы выполнять полезные вычисления.